Lesson 4

Linux and Softwares

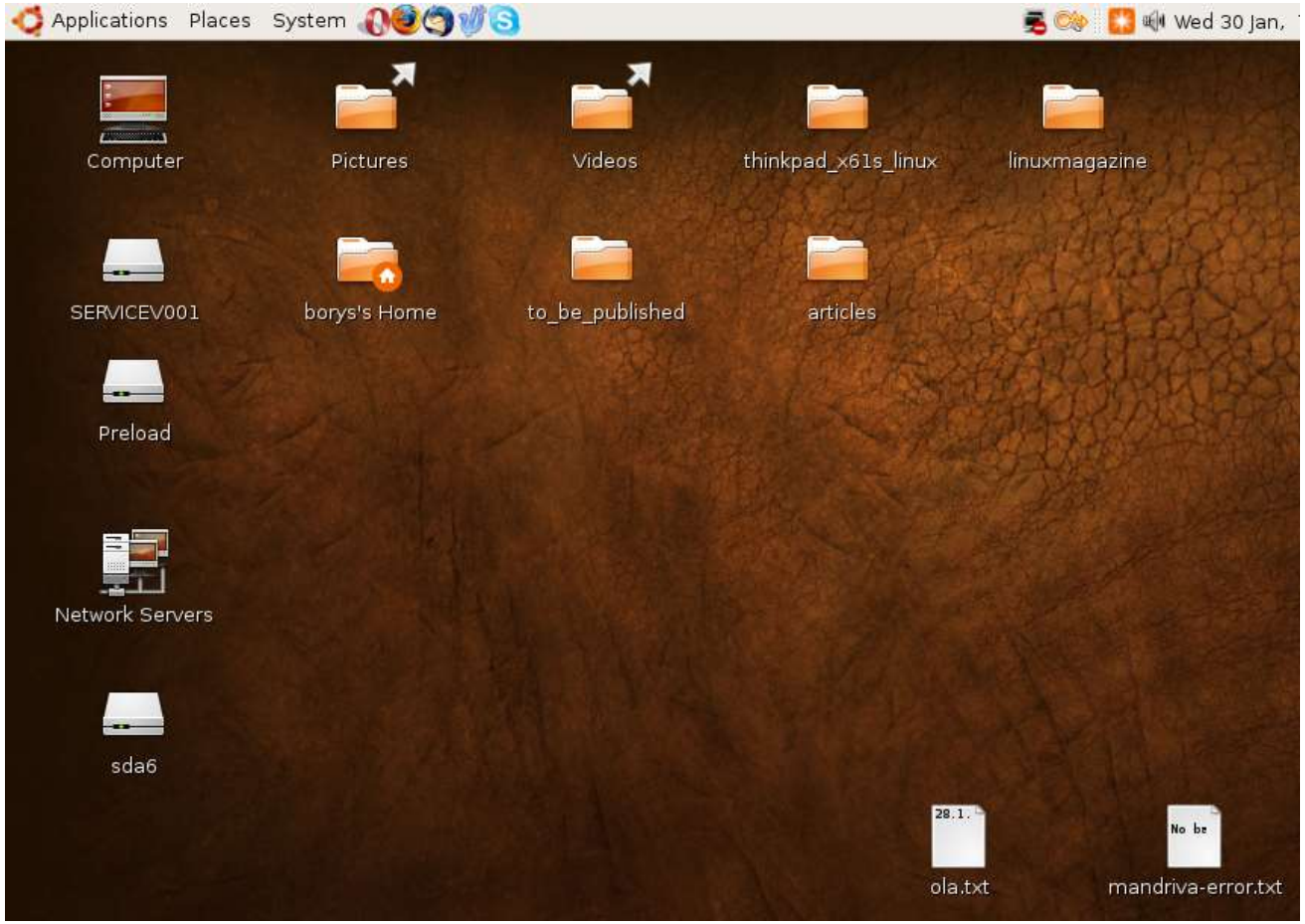
Linux

- **Linux**: A kernel for a Unix-like operating system.
  - commonly seen/used today in servers, mobile/embedded devices, ...
- **GNU**: A "free software" implementation of many Unix-like tools
  - many GNU tools are distributed with the Linux kernel
- **distribution**: A pre-packaged set of Linux software
  - examples: Ubuntu, Fedora
- key features of Linux:
  - **Open-source software**: source code can be downloade
  - **free to use**
  - constantly being improved/updated by the community

Linux Desktop

- X-windows
- window managers
- desktop environments
  - Gnome
  - KDE

- How can I try out Linux?
  - CSE Virtual machine



Things you can do in Linux

- Load the course web site in a browser
- Install and play games
- Play MP3s
- Edit photos
- Office processing- text, spreadsheet, slides etc

Shell

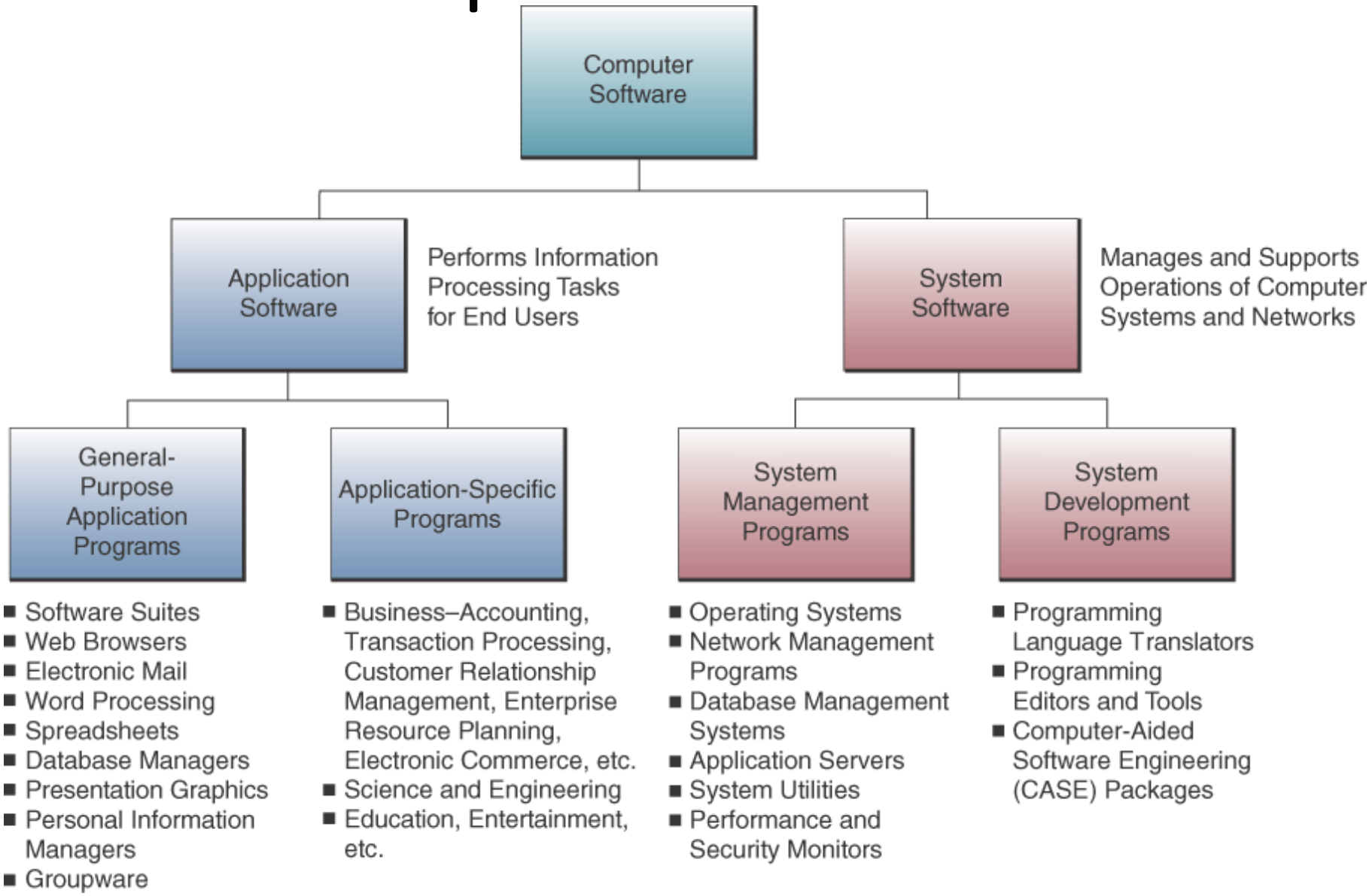
- **shell**: An interactive program that uses user input to manage the execution of other programs.
  - A command processor, typically runs in a text window.
  - User types commands, the shell runs the commands
  - Several different shell programs exist:
    - **`bash`** : the default shell program on most Linux/Unix systems
    - We will use bash
    - Other shells: Bourne, csh, tsch

- Why should I learn to use a shell when GUIs exist?

Why use a shell?

- Why should I learn to use a shell when GUIs exist?
  - faster
  - work remotely
  - programmable
  - customizable
  - repeatable

Computer Softwares



Application Softwares

- General Purpose
  - Programs that perform common information processing jobs for end users; e.g., word processing, spreadsheet
  - Also call productivity packages
- Custom Software
  - Software applications developed within an organization for use by that organization
- Commercial Off-the-Shelf (COTS)
  - Many copies sold
  - Minimal changes beyond scheduled upgrades
  - Purchasers have no control over specifications, schedule, or evolution, and no access to source code or internal documentation
  - Product vendor retains the intellectual property rights of the software
- Open-source Software
  - Developers collaborate on the development of an application using programming standards which allow anyone to contribute to the software
  - As each developer completes a project, the application code becomes available and free to anyone who wants it

Business Application Software

- Function-Specific Application Software
  - Thousands of these packages support specific applications of end users
  - **Examples**: customer relationship management, enterprise resource planning, supply chain management, Web-enabled electronic commerce

Software Suites, Integrated Packages

- Most widely used productivity packages are bundled together as software suites
- Advantages
  - Cost less than buying individual packages
  - All have similar GUI
  - Work well together
- Disadvantages
  - All features not used
  - Takes a lot of disk space (bloatware)

Open Source Software vs. Free Software

- For many, free software and open source software are synonyms.
- The frequently used abbreviation **FOSS** for Free and Open Source Software emphasizes this commonality.
- **FLOSS** for Free/Libre and Open Source Software is another popular term, which unmistakably emphasizes the idea of freedom also for other languages other than English.
- However, if one considers the origin and development of both terms, a differentiation is worthwhile.

Free Software

- The term **free software** with the definition of the described four freedoms goes back to Richard Stallman and the GNU project founded by him in 1985 — almost 10 years before the emergence of Linux.
- The name “**GNU is Not Unix**” describes the intention with a wink of the eye: GNU started as an initiative to develop the operating system Unix from scratch, to make it available to the general public and to improve it continuously with the general public.
- The openness of the source code was merely a technical and organizational necessity for this, but in its self-image the free software movement is still a social and political movement.

Open Source Software

- With the success of Linux, the collaborative possibilities of the Internet, and the thousands of projects and companies that emerged in this new software cosmos, the social aspect increasingly receded into the background.
- The openness of the source code itself changed from a technical requirement to a defining feature: **as soon as the source code was visible**, the software was considered **“open source”.**
- The social motives gave way to a more pragmatic approach to software development.

Licences

- The license under which the software is available determines which conditions a software is subject to with regard to <u>use</u>, <u>distribution</u> and <u>modification</u>.
- And because requirements and motives can be very different, countless different licenses have been created in the FOSS area.
- Due to the much more fundamental approach of the free software movement, it is not surprising that it does not recognize many open source licenses as “free” and therefore rejects them.
- Conversely, this is hardly the case due to the much more pragmatic open source approach.

Software Licensing

- All COTS and ASP software is licensed
  - Involves the underlying..
    - Intellectual property rights
    - Copyright
    - Trademark
    - Trade secrets
  - Also involves traditional contract law, including Uniform Commercial Code (UCC)
- You don’t buy software
  - You buy a license to use the software
  - Licensed to protect the vendor’s property rights

Revision Questions

- Define Software.
- What does GNU stand for?
- State four different shells that can be used in Linux
- Distinguish between system software and application software and give an example for each.
- Discuss the “four essential freedoms” with regard to free softwares.
- Why should I learn to use a shell when GUIs exist?